EM Algorithm

AIM

To implement the Expectation-Maximization (EM) algorithm in Machine Learning and understand how it estimates the parameters of a probabilistic model when the dataset contains hidden or unobserved variables. The implementation demonstrates the iterative process of estimating latent variables and updating model parameters until convergence.

Description

The Expectation-Maximization (EM) algorithm is an iterative optimization technique used to estimate unknown parameters in models containing latent variables. It alternates between the Expectation (E) step, which estimates hidden variables, and the Maximization (M) step, which updates model parameters. The process continues until convergence.

Code

```
import matplotlib.pyplot as plt
from sklearn import datasets
from sklearn.cluster import KMeans
import pandas as pd
import numpy as np
from sklearn import preprocessing
from sklearn.mixture import GaussianMixture

iris = datasets.load_iris()

X = pd.DataFrame(iris.data)
X.columns = ['Sepal_Length', 'Sepal_Width', 'Petal_Length', 'Petal_Width']

y = pd.DataFrame(iris.target)
y.columns = ['Targets']

model = KMeans(n_clusters=3, random_state=42)
model.fit(X)

plt.figure(figsize=(14, 14))

colormap = np.array(['red', 'lime', 'black'])
```

```
plt.subplot(2, 2, 1)
plt.scatter(
    X.Petal_Length,
    X.Petal_Width,
    c=colormap[y.Targets],
    s=40
)
plt.title('Real Clusters')
plt.xlabel('Petal Length')
plt.ylabel('Petal Width')

plt.subplot(2, 2, 2)
plt.scatter(
    X.Petal_Length,
    X.Petal_Width,
    c=colormap[model.labels_],
    s=40
)
plt.title('K-Means Clustering')
plt.xlabel('Petal Length')
plt.ylabel('Petal Width')

scaler = preprocessing.StandardScaler()
scaler.fit(X)

xsa = scaler.transform(X)
xs = pd.DataFrame(xsa, columns=X.columns)

gmm = GaussianMixture(n_components=3, random_state=42)
gmm.fit(xs)

gmm_y = gmm.predict(xs)

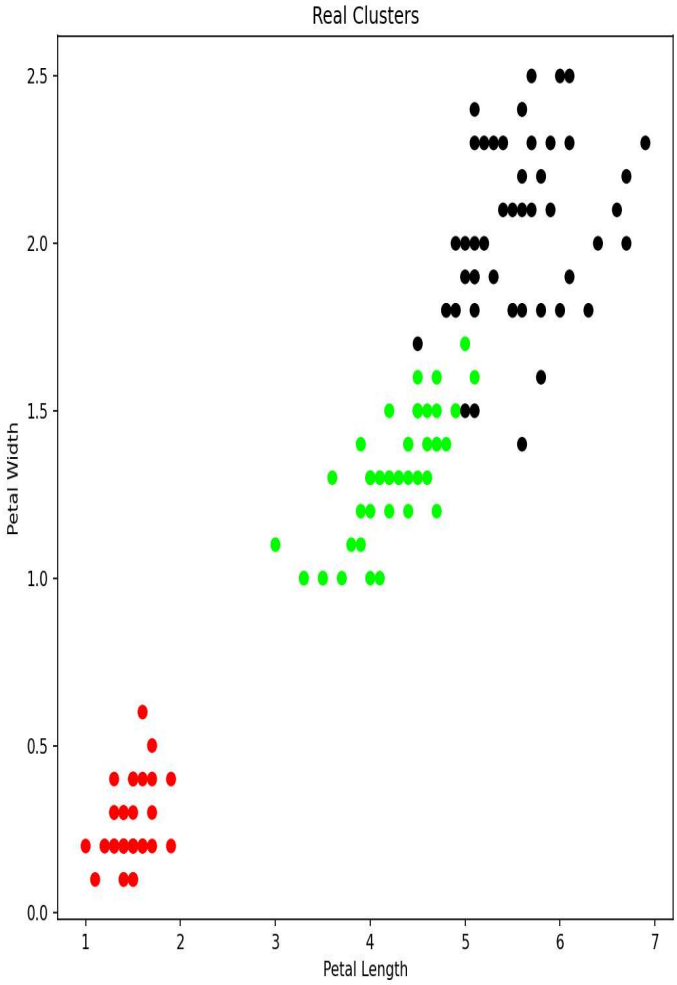
plt.subplot(2, 2, 3)
plt.scatter(
    X.Petal_Length,
    X.Petal_Width,
    c=colormap[gmm_y],
    s=40
)
plt.title('GMM Clustering')
plt.xlabel('Petal Length')
plt.ylabel('Petal Width')

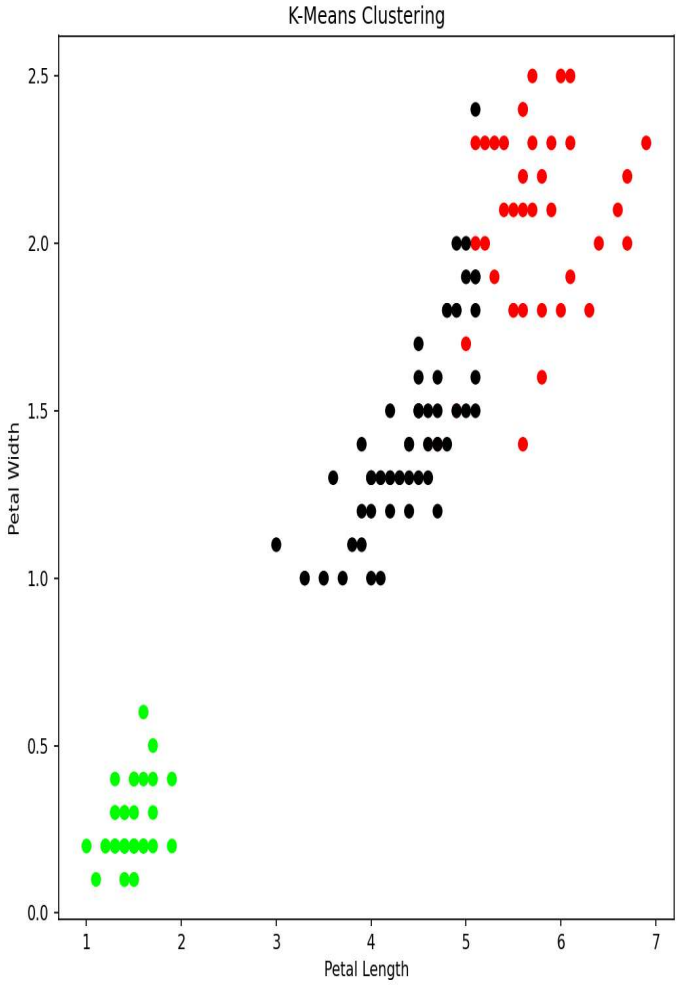
plt.tight_layout()
```

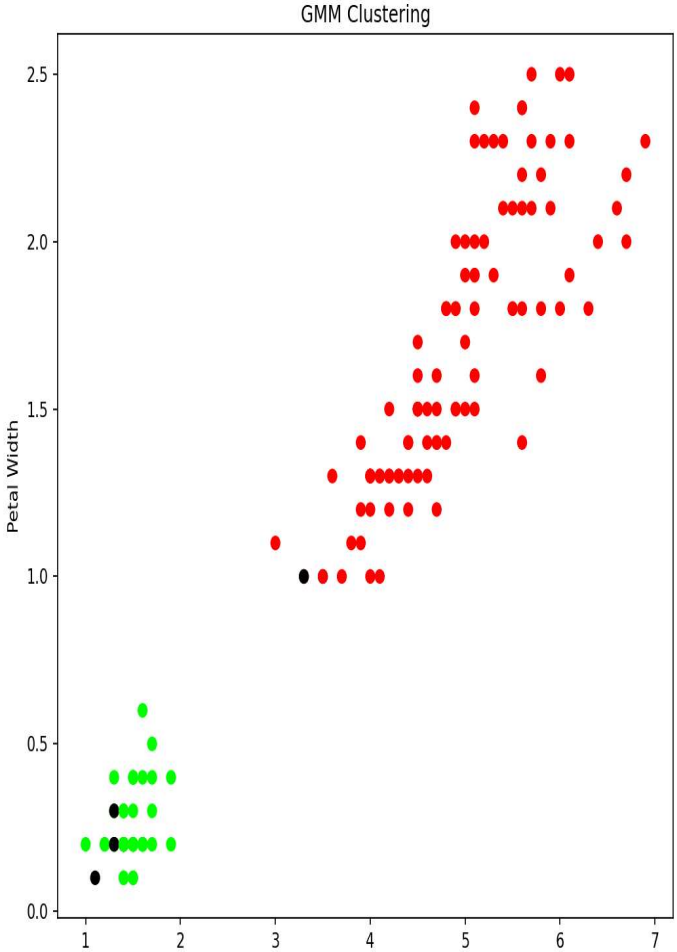
```
plt.show()

print('Observation: The GMM using EM algorithm based clustering matched the true
labels more closely than the Kmeans.')
```

Output


Real Clusters
Petal Width
Petal Length
0.0
0.5
1.0
1.5
2.0
2.5
1
2
3
4
5
6
7


K-Means Clustering
Petal Width
Petal Length
0.0
0.5
1.0
1.5
2.0
2.5
1
2
3
4
5
6
7


GMM Clustering
Petal Width
0.0
0.5
1.0
1.5
2.0
2.5
1
2
3
4
5
6
7

Petal Length